

React + TypeScript: Основы (60-минутная лекция)

1. Введение и настройка окружения (10 минут)

- **Цель лекции:** познакомиться с основами разработки на **React** с использованием **TypeScript**. Рассмотрим, как настроить проект, создадим простое приложение и изучим ключевые концепции (JSX, компоненты, пропсы, состояние, хуки, условный рендеринг, списки, обработка событий, маршрутизация).
- **Что нужно заранее:** установлен Node.js (желательно актуальной версии LTS). Мы будем использовать современный способ запуска React-проекта – через инструмент сборки **Vite** (более быстрый альтернативный вариант Create React App). TypeScript будет сразу интегрирован в проект.
- **Создание проекта:**
- **Вариант 1: Create React App (CRA).** Выполните команду создания приложения с шаблоном TypeScript:

```
npx create-react-app my-app --template typescript
```

Это создаст новый проект React с поддержкой TypeScript ¹ (шаблон автоматически настроит TypeScript). После установки зависимостей проект можно запустить: `npm start` (dev-сервер стартует на <http://localhost:3000>).

- **Вариант 2: Vite.** Выполните команду и выберите шаблон `react-ts` для React+TypeScript:

```
npm create vite@latest my-app -- --template react-ts
```

² После ответа на вопросы Vite установит зависимости. Запустите разработческий сервер:

```
cd my-app
npm install
npm run dev
```

Проект соберется и откроется на <http://localhost:5173> ³. Vite обеспечивает сверхбыструю перезагрузку модулей (HMR) и минимальную конфигурацию.

- **Структура проекта:** В папке проекта расположены: `index.html` (основной HTML с контейнером `<div id="root">`), папка `src` (исходный код приложения), внутри `src` – файл входной точки (`index.tsx` или `main.tsx`, где происходит рендер React в DOM), а

также основной компонент приложения `App.tsx`. Мы будем работать главным образом с файлом `App.tsx` и создавать новые компоненты в папке `src`.

Примечание: В лекции мы используем **Vite** для демонстраций (за счет скорости запуска и современной конфигурации). Однако все рассмотренные концепции полностью применимы и при использовании Create React App.

2. JSX и компоненты: основы (10 минут)

- **JSX:** JSX (JavaScript XML) – это синтаксис, который позволяет писать разметку, похожую на HTML, прямо в JavaScript/TypeScript. Браузеры не понимают JSX напрямую, но сборщик (Webpack, Vite) преобразует JSX в вызовы `React.createElement`. Используя JSX, мы можем описывать UI декларативно. Например, `<h1>Hello, React!</h1>` внутри JSX-кода вернет React-элемент `<h1>` со значением "Hello, React!".
- **Компоненты:** В React интерфейс строится из компонентов. Компонент – это самостоятельный блок UI, обычно реализованный как **функция** (функциональный компонент) или класс (классовый компонент). Мы будем использовать **функциональные компоненты** и хуки, так как это современный подход. Каждый компонент возвращает JSX-разметку. Компоненты можно вкладывать друг в друга, создавая иерархию интерфейса.
- **Пример простого компонента:** создадим компонент `HelloMessage` и используем его в основном компоненте `App` для вывода приветствия:

```
// Компонент, возвращающий элемент заголовка
function HelloMessage() {
  return <h1>Hello, React!</h1>;
}

// Основной компонент приложения
function App() {
  return (
    <div>
      {/* Используем наш компонент внутри JSX */}
      <HelloMessage />
    </div>
  );
}

export default App;
```

Что здесь происходит: мы определили функцию `HelloMessage` – это компонент, который при вызове возвращает `<h1>Hello, React!</h1>`. В `App` внутри JSX мы пишем `<HelloMessage />` – так мы вставляем результат выполнения нашего компонента. Каждый компонент **должен** начинаться с заглавной буквы (React отличает пользовательские компоненты от встроенных тегов HTML по первой букве). Мы обернули всё содержимое `App` в один корневой `<div>` – в JSX

компонент должен возвращать один корневой элемент (в случае необходимости вернуть несколько соседних элементов без обёртки, можно использовать фрагмент `<> ... </>`).

- **Отрендерить компонент в DOM:** Фреймворк React монтирует компонент `App` в DOM-элемент с id `root` (эта логика уже написана в `main.tsx` / `index.tsx` после создания проекта – там вызывается `createRoot(...).render(<App />)`). Таким образом, всё, что возвращает `App`, появится в браузере внутри элемента `<div id="root">` из `index.html`.

3. Пропсы и состояние (10 минут)

Теперь перейдем к тому, как компоненты обмениваются данными и хранят внутренние данные.

- **Props (пропсы):** "Props" (от *properties*) – это входные данные компонента, передаваемые **извне** (родительским компонентом). Когда мы пишем `<MyComponent someProp={42} />`, мы передаем значение 42 в компонент `MyComponent`. Компонент принимает пропсы как параметр (обычно это объект). В функциональных компонентах пропсы доступны через параметр функции. **Важно:** пропсы **только для чтения** – компонент не должен их изменять, они приходят «сверху». Если нужно изменить отображение – либо меняются пропсы от родителя, либо используется собственное состояние.
- **Пример пропсов:** допустим, нам нужен компонент `Greeting`, который выводит приветствие пользователю по имени. Имя передается как проп. В TypeScript мы сперва описываем интерфейс пропсов (состав и типы), затем используем его в компоненте:

```
interface GreetingProps {
  name: string; // описываем проп "name" типа string
}

function Greeting({ name }: GreetingProps) {
  return <p>Привет, {name}!</p>;
}

function App() {
  return (
    <div>
      <Greeting name="Алиса" />
      <Greeting name="Боб" />
    </div>
  );
}
```

В этом примере компонент `Greeting` ожидает проп `name` (строка) и рендерит параграф с приветствием. Мы используем деструктуризацию `{ name }` в параметрах функции для удобства. В `App` мы дважды используем `<Greeting>` с разными именами – в каждом случае компонент получит свое значение пропса `name` и выведет персонализированное сообщение. Если попробовать передать неправильный тип пропса (например, число вместо строки), TypeScript

выдаст ошибку на этапе компиляции – это полезно для отлова ошибок еще до запуска приложения.

- **State (состояние):** В отличие от пропсов, которые приходят извне, **состояние** компонента – это внутренние, изменяемые данные, которые компонент **может менять сам**. Когда состояние меняется, компонент **перерисовывается** (React обновляет DOM под капотом, показывая актуальные данные). В функциональных компонентах состояние управляется через **хуки** (например, `useState`).
- **Хук `useState`:** это встроенная функция, которая позволяет компоненту хранить состояние между рендерами. При вызове `useState(initialValue)` она возвращает массив из двух элементов: текущее значение состояния и функция для его обновления. Вызов функции обновления инициирует повторный рендер компонента с новым состоянием.
- **Пример состояния:** реализуем простой счетчик:

```
import { useState } from 'react';

function Counter() {
  // Инициализируем состояние count значением 0
  const [count, setCount] = useState(0);

  return (
    <div>
      <p>Счетчик: {count}</p>
      <button onClick={() => setCount(count + 1)}>Увеличить</button>
    </div>
  );
}

function App() {
  return <Counter />;
}
```

Здесь мы использовали хук `useState(0)`, передав начальное значение `0`. React вернул массив, который мы раздеструктурировали: `count` – текущее значение (инициально `0`), `setCount` – функция, изменяющая значение. При нажатии кнопки вызывается `setCount(count + 1)` – React обновит состояние `count` и перерисует компонент `Counter`, отобразив новое значение. Состояние компонентов изолировано – каждый экземпляр `Counter` хранил бы свой `count`.

Важно: изменять состояние нужно **только** через функцию, полученную от `useState` (напрямую присваивать `count = ...` нельзя, это не изменит внутреннего состояния React). Также обновление состояния происходит асинхронно и может быть объединено, поэтому полагаться на старое `count` при нескольких вызовах подряд не стоит – в таких случаях используют функцию: `setCount(prevCount => prevCount + 1)`.

4. Базовые хуки: useState и useEffect (10 минут)

- Мы уже рассмотрели хук `useState`, который используется для управления состоянием. Он часто применяется для хранения простых данных: чисел, строк, булевых флагов, объектов или массивов (в последних случаях обновлять состояние следует через копирование/иммутабельность, например, используя спред-оператор, чтобы React понял, что ссылка изменилась).
- Хук `useEffect`**: позволяет выполнять побочные эффекты в функциональных компонентах. "Побочные эффекты" – это любые действия, которые нужно выполнить в ответ на изменение состояния или пропсов, либо при монтировании/размонтировании компонента: например, загрузка данных с сервера, подписка на веб-сокеты, таймеры, прямое изменение DOM вне React (если необходимо), и пр. `useEffect` принимает функцию-эффект и массив зависимостей. Эта функция будет выполнена **после** рендера компонента (асинхронно), а зависимости определяют, когда именно вызывать эффект:
- Пустой массив зависимостей `[]` – эффект выполнится **один раз** после первого рендера (аналогично `componentDidMount` в классовом подходе).
- Отсутствие второго аргумента – эффект выполняется после **каждого** рендера (каждого изменения состояния или пропсов).
- Наличие списка зависимостей `[a, b, ...]` – эффект будет запускаться после рендера **только при изменении** этих указанных значений (или при первом монтировании).
- Пример `useEffect`**: добавим в наш счетчик эффект, который обновляет заголовок страницы в зависимости от значения счетчика:

```
import { useState, useEffect } from 'react';

function Counter() {
  const [count, setCount] = useState(0);

  // Побочный эффект: обновить заголовок окна при изменении count
  useEffect(() => {
    document.title = `Count: ${count}`;
  }, [count]); // срабатывает только когда меняется count

  return (
    <div>
      <p>Счетчик: {count}</p>
      <button onClick={() => setCount(count + 1)}>Увеличить</button>
    </div>
  );
}
```

Когда компонент `Counter` монтируется, эффект сразу выполнится (установит `document.title = "Count: 0"`). Каждый раз при обновлении `count`, React после отрисовки вызовет наш эффект и обновит заголовок вкладки на актуальное значение. Мы указали `[count]` как

зависимость – значит, если значение `count` не изменилось при очередном рендере, эффект повторно не выполняется.

Примечание: `useEffect` можно использовать и для **очистки** ресурсов – если функция, переданная в `useEffect`, возвращает другую функцию, то последняя будет вызвана при размонтировании компонента или перед следующим выполнением эффекта. Это полезно для отмены таймеров, отписки от подписок и т.д. (пример: `return () => clearTimeout(timerId)` внутри эффекта).

- **Итого про хуки:** `useState` и `useEffect` – самые базовые и часто используемые хуки. Они покрывают сценарии внутреннего состояния и реакции на изменения. В React есть и другие встроенные хуки (например, `useContext`, `useReducer`, `useMemo`, `useRef`...), но их можно изучить позже. Для начала двух будет достаточно, чтобы понимать основу функционального подхода.

5. Типизация компонентов с TypeScript (5 минут)

- Использование TypeScript дает большие преимущества при разработке на React: появляется статическая проверка типов, автодополнение в IDE, уменьшается количество ошибок. Рассмотрим, как типизировать наши компоненты и хуки:
- **Типизация пропсов:** мы уже это применили – описываем интерфейс (или тип) пропсов и указываем его в определении функционального компонента. Например, `interface GreetingProps { name: string }`, а компонент принимает `{ name }: GreetingProps`. Благодаря этому, если попытаться передать не тот тип (например, `<Greeting name={123} />`), TypeScript предупредит об ошибке.
- **Типизация состояния:** хук `useState` — обобщенная функция, она выводит тип состояния из переданного начального значения. В примере `useState(0)` тип состояния будет `number`. Можно указать тип явно, например: `const [value, setValue] = useState<string>("")` – здесь мы сказали, что состояние будет строковым. Явно указывать тип нужно, если начальное значение не определяет полный тип (например, `useState<MyType | null>(null)`).
- **Типизация компонентов:** Можно явно указать тип для самого компонента, используя тип `React.FC<Props>`. Например:

```
import { FC } from 'react';
interface ButtonProps { text: string; }
const Button: FC<ButtonProps> = ({ text }) => <button>{text}</button>;
```

Однако, в современном TS это необязательно – зачастую достаточно объявить функцию через `function` или `const` с указанием типов ее параметров (как мы делали с пропсами), и TypeScript выведет возвращаемый тип сам (JSX-элемент).

- **Типы элементов JSX:** при работе с DOM-элементами у нас есть готовые типы событий и элементов. Например, `React.MouseEvent<HTMLButtonElement>` – тип события клика по кнопке, `React.ChangeEvent<HTMLInputElement>` – тип события изменения поля ввода, и т.д. (подробнее о событиях – в следующем разделе).
- **Пример преимущества TS:** если в компоненте `Greeting` мы типизировали пропсы, то IDE подскажет, какие пропсы ожидаются. Попытка использовать несуществующий проп `<Greeting firstname="Ann" />` или проп пропустить обязательный проп приведет к

ошибке компиляции. Строгая типизация делает код надежнее ⁴ и ускоряет разработку, так как многие ошибки выявляются сразу в редакторе.

- **Совет:** на первых порах, если не знаете, как типизировать какую-то сущность, можно временно использовать тип `any` (он отключает проверку типа) ⁵, но старайтесь по возможности описывать реальные типы – так вы быстрее освоите TypeScript. Полезный ресурс – официальные шаблоны и подсказки по React+TS (React + TypeScript Cheatsheets).

6. Обработка событий (5 минут)

- Реактивные приложения предполагают взаимодействие с пользователем, поэтому важно уметь обрабатывать события: клики, ввод текста, отправку форм и т.д. В React события именуются в **camelCase** (например, `onClick`, `onChange`) и передаются в JSX как атрибуты, значением которых является функция-обработчик. Внутри обработчика мы можем вызывать `setState` для изменения состояния или выполнять другие действия.
- **Синтетические события:** React использует свою систему событий (Synthetic Events), которая оборачивает нативные события браузера для кросс-браузерной совместимости. Тем не менее, вы взаимодействуете с ними почти так же, как с обычными DOM-событиями. Например, обработчик клика получает объект события `MouseEvent`, на котором доступны свойства типа координат клика, целевого элемента и др.
- **Пример обработчика клика:** Создадим компонент кнопки логина, который по нажатию вызывает функцию (например, имитация логина):

```
import React from 'react';

interface LoginButtonProps {
  onLogin: () => void; // пропсы могут передавать и функции
}

function LoginButton({ onLogin }: LoginButtonProps) {
  // Обработчик клика
  function handleClick(event: React.MouseEvent<HTMLButtonElement>) {
    console.log('Clicked button:', event.currentTarget);
    onLogin(); // вызываем переданный колбэк
  }

  return <button onClick={handleClick}>Войти</button>;
}

// Использование:
function App() {
  const login = () => { alert("Авторизация..."); };
  return <LoginButton onLogin={login} />;
}
```

Здесь `LoginButton` ждет проп `onLogin` (функция без параметров). При клике мы получаем объект события `event` типа `React.MouseEvent` для `HTMLButtonElement`. Мы вывели в консоль

текущий целевой элемент (кнопку) и вызвали функцию `onLogin` (которая была передана из `App`). Если бы нам не нужен был объект `event`, можно было просто написать `onClick={onLogin}` при использовании кнопки – но в данном случае мы показали, как явно объявить свой обработчик и воспользоваться объектом события. TypeScript автоматически знает тип события по типу элемента, на котором висит обработчик (если бы мы использовали `onChange` на `<input>`, обработчик имел бы тип `React.ChangeEvent<HTMLInputElement>` и т.д.).

- **Другие события:** Аналогично обрабатываются другие события: `onChange` для ввода (мы могли бы обновлять состояние по `event.target.value`), `onSubmit` для отправки формы, `onKeyDown`, `onMouseEnter` и прочие – React покрывает большинство DOM-событий. Благодаря TS, IDE может показать ожидаемый тип события: например, если поставить курсор на атрибут `onFocus`, можно увидеть, что обработчик должен принять `React.FocusEvent<Element>`.

7. Рендеринг списков (5 минут)

- Часто необходимо отобразить набор однотипных данных – список элементов. В React для этого обычно используют метод массива `.map()`, который преобразует каждый элемент массива в JSX-элемент. При рендеринге списков **очень важно** указывать свойство `key` для корневого элемента внутри списка – ключ должен уникально идентифицировать элемент среди соседей. Ключи помогают React эффективно обновлять список при изменениях (сопоставляя элементы списка между рендерами по ключам, React избегает лишних перерисовок) ⁶.

- **Пример списка:** Отобразим список фруктов:

```
function FruitsList() {
  const fruits = ["Apple", "Banana", "Cherry"];

  return (
    <ul>
      {fruits.map(fruit => (
        <li key={fruit}>{fruit}</li>
      ))}
    </ul>
  );
}

function App() {
  return <FruitsList />;
}
```

Компонент `FruitsList` берёт массив строк `fruits` и при помощи `map` возвращает для каждого элемента `<li>` с текстом фрукта. Атрибут `key={fruit}` использован в данном примере – здесь в роли ключа выступает сам текст (предполагаем, что названия уникальны). В реальных приложениях вместо текста лучше использовать уникальный идентификатор

(например, `id` объекта). **Никогда не используйте индекс массива в качестве ключа**, если порядок элементов может меняться – это может привести к ошибкам при обновлении списка.

- **Динамические списки:** Списки могут рендериться на основе состояния или пропсов. Например, мы можем хранить массив объектов в состоянии и отображать его. При добавлении/удалении элементов (через изменение состояния) React перерисует список, добавив или убрав соответствующие узлы. Всегда следите за правильностью ключей, особенно при удалении – ключи должны оставаться привязанными к одним и тем же данным между рендерами.

8. Условный рендеринг (5 минут)

- Интерфейс часто должен отображаться по-разному в зависимости от условий. В React условный рендеринг реализуется с помощью обычных условных конструкций JavaScript:
- Можно использовать тернарный оператор в JSX: `{condition ? <TrueComponent /> : <FalseComponent />}` – отобразится первый вариант, если условие истинно, иначе второй.
- Можно использовать логическое И `&&` для короткого синтаксиса: `{condition && <SomeComponent />}` – отобразится `<SomeComponent />` только если `condition === true` (иначе JSX просто ничего не выведет в этом месте).
- Также можно вынести условие за пределы JSX (например, вычислить переменную) или использовать `if` внутри тела компонента (но нельзя напрямую писать `if` внутри самого JSX-разметки, только внутри `{...}` как выражение, либо до `return`).
- **Пример условного рендеринга:** Создадим компонент, который показывает либо приветствие, либо приглашение залогиниться в зависимости от статуса пользователя:

```
function UserInfo({ isLoggedIn }: { isLoggedIn: boolean }) {
  return (
    <div>
      {isLoggedIn ? (
        <p>Welcome back!</p>
      ) : (
        <p>Please log in.</p>
      )}
    </div>
  );
}

// Использование:
function App() {
  const userLogged = false;
  return <UserInfo isLoggedIn={userLogged} />;
}
```

Здесь мы использовали тернарный оператор прямо внутри JSX. Если `isLoggedIn` истинно, будет выведено `<p>Welcome back!</p>`, иначе – `<p>Please log in.</p>`. Обратите внимание: оба варианта должны иметь общий родитель (`<div>` у нас) или быть обернуты

фрагментом `<>...</>`, поскольку JSX требует единого возвращаемого узла. Проп `isLoggedIn` имеет тип `boolean`, TypeScript проследит, чтобы мы передали корректное значение.

- **Еще пример (логическое И):** иногда удобен синтаксис `&&` для простых случаев. Например, если нужно отобразить сообщение об ошибке только когда есть текст ошибки:

```
{errorMessage && <div className="error">Error: {errorMessage}</div>}
```

Если `errorMessage` пустая строка (`falsy`), то выражение вернет `false` и див не будет отрендерен; если там есть строка, React отрендерит элемент. Будьте осторожны с значениями, которые могут быть числом 0 – они тоже `falsy` и могут нечаянно скрыть элемент.

- **Условные переходы:** Можно комбинировать условия и списки: например, рендерить список элементов или сообщение "нет данных", если список пуст:

```
{items.length > 0 ? <ItemsList items={items} /> : <p>No items found.</p>}
```

Такой подход часто применяется для отображения загрузки, сообщений об отсутствии данных, разных видов представления в зависимости от состояния приложения.

9. Маршрутизация: основы React Router (10 минут)

- **Что такое маршрутизация:** В SPA (Single Page Application) приложения на React обычно представляют собой одну страницу, которая динамически подгружает разные представления (вьюхи) по мере взаимодействия, вместо полной перезагрузки страницы. **React Router** – это стандартная библиотека для организации маршрутизации в React-приложениях. Она позволяет определить **маршруты** (routes) – соответствие между URL-путями и компонентами, и навигировать пользователя между разными представлениями без перезагрузки страницы. Проще говоря, с React Router можно разделить приложение на несколько экранов (страниц) и переключаться между ними по URL, оставаясь в рамках одного React-приложения.

- **Установка React Router:** Добавим пакет маршрутизатора в проект (если не был добавлен):

```
npm install react-router-dom
```

(`react-router-dom` – это версия React Router для веб-браузера). После установки можно использовать компоненты маршрутизатора в нашем приложении.

- **Базовые компоненты Router:** Чтобы включить маршрутизацию, нужно обернуть приложение в компонент `<BrowserRouter>` (делает историю браузера доступной для внутренних компонентов). Затем определяются маршруты с помощью `<Routes>` и `<Route>`. Также используются ссылки `<Link>` (или `<NavLink>`) вместо обычных `<a>`

для навигации внутри приложения, чтобы перехватывать клики и предотвращать перезагрузку страницы.

- **Пример маршрутизации:** Реализуем простейшую структуру с двумя страницами – "Home" и "About":

```
import { BrowserRouter, Routes, Route, Link } from 'react-router-dom';

function HomePage() {
  return <h2>Home Page</h2>;
}

function AboutPage() {
  return <h2>About Us</h2>;
}

function App() {
  return (
    <BrowserRouter>
      { /* Навигационное меню */ }
      <nav style={{ marginBottom: '1em' }}>
        <Link to="/">Home</Link> | <Link to="/about">About</Link>
      </nav>

      { /* Определение маршрутов */ }
      <Routes>
        <Route path="/" element={<HomePage />} />
        <Route path="/about" element={<AboutPage />} />
      </Routes>
    </BrowserRouter>
  );
}

export default App;
```

Разберем этот пример: - Мы импортировали компоненты из `react-router-dom`. - Обернули всё приложение `<App>` в `<BrowserRouter>` - это необходимо сделать **единожды** на верхнем уровне (обычно в файле `main.tsx` или прямо в `App.tsx`, как у нас, если `App` считается корневым компонентом приложения). `BrowserRouter` включает механизм слежения за историей URL. - Внутри, перед определением маршрутов, мы добавили простой блок `<nav>` с двумя ссылками `<Link>`. Атрибут `to` определяет, на какой путь переходить. Эти ссылки заменяют обычные `<a>` - при клике на `<Link>` React Router обновит адресную строку и отрисует нужный компонент, **не перезагружая страницу**. - Блок `<Routes>` содержит список `<Route>` - каждый маршрут указывает путь (`path`) и React-компонент, который нужно отобразить в этом случае (`element`). - Когда URL совпадает с `path="/"`, внутри `<Routes>` будет рендериться `<HomePage />`. - Когда URL = `/about`, будет рендериться `<AboutPage />`. - Компоненты `HomePage` и `AboutPage` здесь для простоты определены прямо внутри файла. В реальном приложении их код мог бы быть в отдельных файлах, импортированных наверху. Они просто отображают заголовки для демонстрации.

Как это работает: при первом открытии приложения на URL `/` отрисовывается `HomePage`. При клике на ссылку "About", адрес в браузере изменится на `/about` (но страницы не перезагрузится), React Router заметит соответствие маршруту `/about` и вместо `HomePage` покажет компонент `AboutPage`. Состояние React (включая, например, данные в компонентах) не теряется при переходах, так как это одна SPA – меняется только отображаемый компонент внутри `<Routes>`.

- **Дополнительные возможности:** React Router предоставляет много функций: *параметры маршрутов* (например, `/product/:id`), *вложенные маршруты* (для layouts, когда одна страница содержит область, меняющуюся по подпутям), *редиректы*, *защищенные маршруты* (требующие авторизации) и пр. В нашем часовом занятии мы лишь успеваем упомянуть основу – определение маршрутов и навигацию. Важно понимать, что маршрутизация – это ключ к структуре приложения, позволяющей пользователю перемещаться между разными экранами. После лекции вы сможете самостоятельно изучить документацию React Router для продвинутых случаев.
- **Маршрутизация и TypeScript:** типизация маршрутов обычно не вызывает трудностей. Иногда полезно определить типы для параметров (React Router v6+ предоставляет хук `useParams` для получения параметров из URL, их можно типизировать). В простых случаях, как наш, можно довериться проверкам at runtime. Функции навигации (`useNavigate`) и др. тоже имеют типы, которые IDE подскажет при импорте.
- **Итог по React Router:** подключение Router несложное – обернуть приложение в `BrowserRouter`, задать `Routes` и `Route`. Таким образом, за ~10 минут мы сделали мини-приложение с двумя страницами. Это завершает нашу основную часть лекции.

Заключение (итоги)

Мы рассмотрели весь запланированный материал. За ~60 минут мы: 1. Настроили среду и создали новый React-проект с TypeScript (через Vite, упомянули CRA как альтернативу). 2. Разобрались с JSX и компонентами – строительными блоками React-приложения. 3. Познакомились с передачей данных через **пропсы** и хранением внутреннего **состояния** компонента. 4. Научились использовать базовые **хуки** `useState` и `useEffect` для состояния и побочных эффектов. 5. Увидели, как **TypeScript** усиливает React: типизация пропсов, состояний, событий. 6. Поговорили про обработку событий в React и увидели примеры с кнопками. 7. Реализовали **рендеринг списков** из массива с использованием `Array.map` и узнали про важность уникальных ключей. 8. Реализовали **условный рендеринг** различных элементов UI на основе булевых условий. 9. Добавили простейшую **маршрутизацию** с помощью React Router, чтобы переключаться между страницами без перезагрузки.

Каждая из этих тем – основа для более глубокого изучения. Но даже освоив базу, вы уже можете создавать простые приложения на React + TypeScript. Дальше советуем попрактиковаться: попробовать добавить еще один маршрут, сделать форму с управляемым состоянием (контролируемые компоненты), поэкспериментировать с другими событиями. Используйте TypeScript подсказки при разработке – они помогут писать код правильнее.

Спасибо за внимание! Надеемся, что демо-код и объяснения в этой лекции помогли вам разобраться с фундаментальными концепциями **React** и **TypeScript** в разработке интерфейсов. Удачи в дальнейших экспериментах и разработке!

1 Adding TypeScript | Create React App

<https://create-react-app.dev/docs/adding-typescript/>

2 Что такое Vite и как запускать проекты React через него

<https://purpleschool.ru/knowledge-base/article/vite-js-react>

3 Complete Guide to Setting Up React with TypeScript and Vite (2025) | by Robin Viktorsson | Medium

<https://medium.com/@robinviktorsson/complete-guide-to-setting-up-react-with-typescript-and-vite-2025-468f6556aaf2>

4 5 Использование Typescript с React – руководство для новичков / Хабр

<https://habr.com/ru/companies/otus/articles/456124/>

6 Введение: знакомство с React

<https://ru.legacy.reactjs.org/tutorial/tutorial.html>